

Figure 3: Minimal coarse-grained spreading simulation on a random interaction graph

Code description for GitHub repository (Mevan Rajakaruna)

M. Rajakaruna

Overview

This Python script generates Figure 3 of the manuscript entitled “Feedback polarity controls causal clustering and cascade propensity in stochastic rupture”. The figure illustrates the mean avalanche size $\langle S \rangle$ as a function of the effective loading parameter $\lambda_{\text{eff}} = \alpha k v t_{\text{mat}}$ under two loading protocols: displacement control (positive feedback) and force control with transient unloading (negative feedback). The simulation employs a random Erdős–Rényi interaction graph and a simple branching avalanche model to demonstrate how the local triggering probability p , derived from the two-bond stochastic rupture model, influences collective failure behaviour.

1 Software and libraries

The script requires Python version 3.9 or higher. Numerical array operations are handled by NumPy (version 1.21 or later). The random graph generation and neighbour iteration functions are provided by NetworkX (version 2.6 or later). All figure construction, including line plotting, markers, and file saving, is performed using Matplotlib (version 3.4 or later).

2 Simulation parameters

The following parameters are fixed at the beginning of the script. The number of nodes in the random graph is $N = 2000$. The mean out-degree is $z_{\text{mean}} = 4$, which determines the edge probability as $p_{\text{edge}} = z_{\text{mean}}/(N - 1)$. The number of avalanche trials per parameter value is 1000. The effective loading parameter λ_{eff} is sampled over 15 linearly spaced values from 0.1 to 3.0.

For force control simulations, the following additional parameters are fixed: the unloading fraction $\beta = 0.3$, the load sensitivity parameter $\alpha F = 3.0$ (dimensionless), the baseline rupture rate $r_0 = 0.01$, and the material time $t_{\text{mat}} = 0.05$.

3 Random graph construction

An undirected Erdős–Rényi random graph is generated using `nx.erdos_renyi_graph(N, p_edge)`. Each edge is included independently with probability $p_{\text{edge}} = z_{\text{mean}}/(N - 1)$,

yielding a graph with expected degree $z_{\text{mean}} = 4$. The graph is generated once and reused for all avalanche simulations to maintain consistency across parameter values. No random seed is explicitly fixed, meaning the graph structure and avalanche outcomes will vary across different runs unless a seed is set externally.

4 Avalanche simulation algorithm

The function `run_avalanche(G, p_trigger)` simulates a single avalanche starting from a single randomly selected seed node. The algorithm proceeds as follows. A set `failed` tracks all nodes that have already failed. A set `active` tracks nodes that have failed in the current generation and will attempt to trigger their neighbours. The seed node is added to both `failed` and `active`.

The simulation then enters a while loop that continues as long as `active` is non-empty. For each active node, the function iterates over all its neighbours in the graph. For each neighbour that has not already failed, a random number drawn from a uniform distribution on $[0, 1)$ is compared against the triggering probability p_{trigger} . If the random number is less than p_{trigger} , the neighbour is added to both `new_active` (the set of nodes that will become active in the next generation) and `failed`. After processing all active nodes, the `active` set is replaced by `new_active`, and the loop repeats. The function returns the total number of failed nodes, which is the avalanche size S .

This algorithm implements a simple branching process on a fixed network topology, where each failed node attempts to trigger each susceptible neighbour exactly once, and triggering events are independent Bernoulli trials with probability p_{trigger} .

5 Triggering probability mapping from the rupture model

The local triggering probability p for each loading protocol is mapped from the two-bond stochastic rupture results as follows. Under displacement control, the small- t_{mat} asymptotic expression gives $p \sim P_c \sim \frac{\alpha k v}{2} t_{\text{mat}} = \lambda_{\text{eff}}/2$. The script therefore sets $p_{\text{disp}} = \lambda_{\text{eff}}/2$ for each value of λ_{eff} .

Under force control with transient unloading, the small- t_{mat} asymptotic expression gives $p \sim P_c \sim r_0 e^{\alpha F} e^{-\alpha F \beta} t_{\text{mat}}$. The script computes this as a constant value using the fixed parameters: $p_{\text{force}} = r_0 \cdot \exp(\alpha F) \cdot \exp(-\alpha F \beta) \cdot t_{\text{mat}}$. With the chosen parameters, $p_{\text{force}} = 0.01 \cdot \exp(3.0) \cdot \exp(-0.9) \cdot 0.05 = 0.01 \cdot 20.0855 \cdot 0.4066 \cdot 0.05 \approx 0.00408$. This value remains constant across all λ_{eff} because force control does not involve a pulling rate v .

6 Mean avalanche size calculation

For each value of λ_{eff} in `lambda_vals`, the script performs the following steps. For displacement control, the triggering probability is set to $p_{\text{disp}} = \lambda_{\text{eff}}/2$. One thousand avalanche simulations are performed using `run_avalanche(G, p_disp)`, and the sizes are stored in a list. The mean avalanche size $\langle S \rangle$ for displacement control is computed as the arithmetic mean of these 1000 values. For force control, the constant triggering probability p_{force} is used. One thousand avalanche simulations are performed, and the

mean size is computed similarly. The two resulting sequences of mean avalanche sizes (15 values each) are stored for plotting.

7 Plotting

The figure is created with dimensions 7 inches wide by 5 inches high using `plt.figure(figsize=(7,5))`. The displacement control results are plotted as blue circles connected by solid lines (marker style `'o-'`). The force control results are plotted as orange squares connected by dashed lines (marker style `'s--'`). A vertical dotted gray line is drawn at $\lambda_{\text{eff}} = 2/z_{\text{mean}} = 2/4 = 0.5$, corresponding to the mean-field threshold where the branching ratio $\mathcal{R} = z_{\text{mean}}p = 1$ under the displacement control mapping $p = \lambda_{\text{eff}}/2$. This line is labeled “ $\mathcal{R} = 1$ threshold”.

The x-axis is labeled λ_{eff} (effective loading parameter). The y-axis is labeled $\langle S \rangle$ (mean avalanche size). The legend is placed with no frame and with `fancybox=False` for a clean appearance. The layout is adjusted using `plt.tightlayout()`, and the figure is displayed with `plt.show()`. Unlike the previous figure scripts, this version does not save the figure to disk; it only displays it interactively.

8 Reproducibility

The script does not fix a random seed for either the graph generation or the avalanche simulations. Therefore, the exact numerical values of $\langle S \rangle$ will vary across different runs. The qualitative behaviour, however, remains consistent: displacement control shows increasing $\langle S \rangle$ with λ_{eff} crossing a threshold near $\lambda_{\text{eff}} \approx 0.5$, while force control yields small, approximately constant $\langle S \rangle$ due to the exponentially suppressed triggering probability. For fully reproducible results, a random seed should be set using `np.random.seed()` and `nx.utils.graph_seed()` before graph generation. No external data files are required.

9 Relation to the manuscript

This figure corresponds to Figure 3 in the paper. It provides a minimal network simulation illustrating the branching mapping $\mathcal{R} = zp$, where the local triggering probability p is derived from the two-bond stochastic rupture model. The displacement control case demonstrates growth in mean avalanche size near the mean-field threshold $\mathcal{R} = 1$, consistent with cascading failure. The force control case shows localised avalanches, consistent with the exponential suppression of p and $\mathcal{R} < 1$.

10 Limitations acknowledged in the manuscript

As discussed in the paper, this simulation is heuristic. It assumes independent triggering attempts, ignores stress interactions beyond nearest neighbours, neglects spatial correlations that are known to be important in real fracture, and does not include stress shadows or interaction ranges. The random graph lacks the spatial structure of real materials. This simulation is intended as a qualitative illustration, not a rigorous demonstration that real rupture networks fall into a specific universality class.

License and citation

This code is part of the research by M. Rajakaruna and H. Rajakaruna. If you use this code or the generated figure, please cite the associated manuscript as described in the README.md file in the repository.